

HTTP Status Codes

Status Codes Tell the Client What Happened

2xx — Success

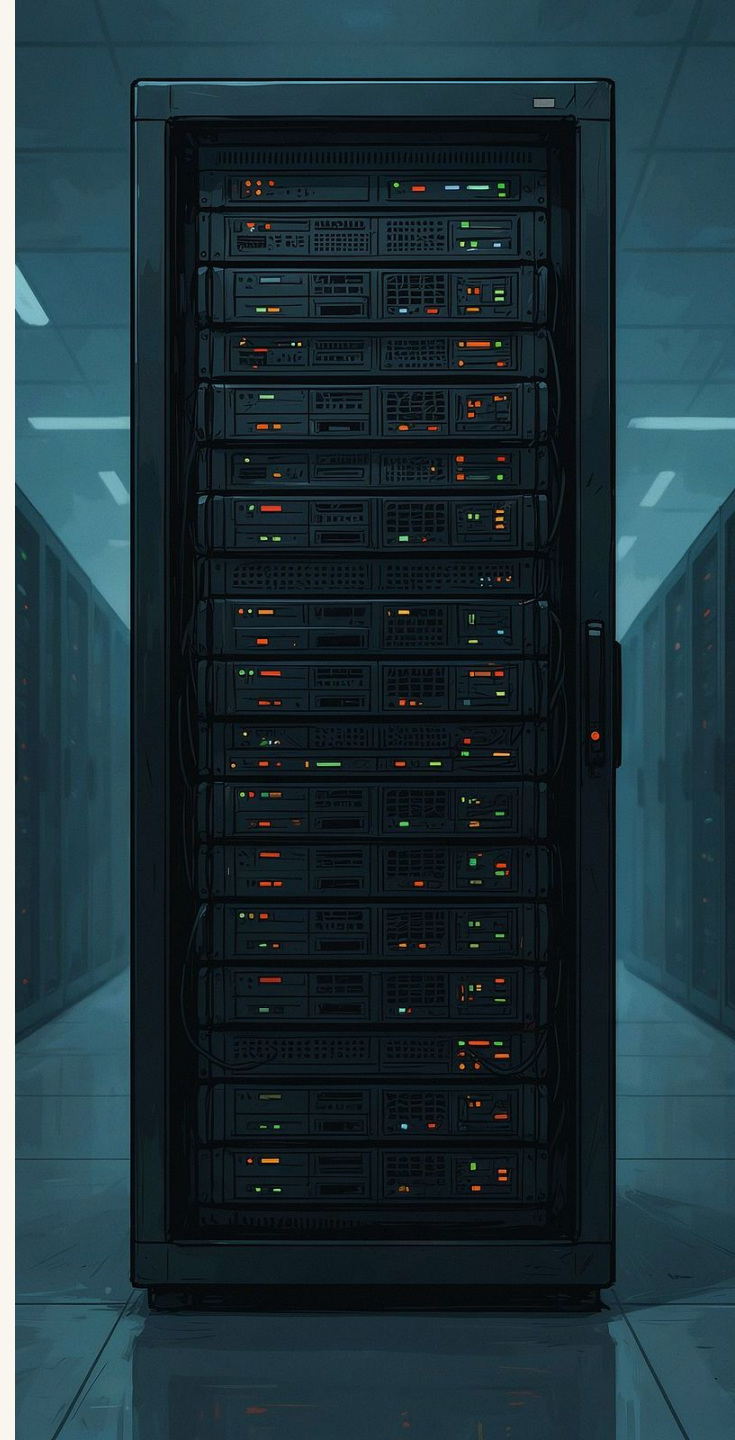
- 200 OK
- 201 Created
- 204 No Content

4xx — Client Error

- 400 Bad Request
- 401 Unauthorized
- 403 Forbidden
- 404 Not Found
- 409 Conflict

5xx — Server Error

- 500 Internal Server Error
- 503 Service Unavailable



Choosing the Correct Status Code

Student API Examples

Scenario	Status Code
Successfully retrieve student	200 OK
Successfully create student	201 Created
Successfully delete student	204 No Content
Student doesn't exist	404 Not Found
Invalid request	400 Bad Request

 Status codes are part of the API contract.

DTOs

Don't Always Expose Your Entity Directly

Database Entity:

Student

API Request:

CreateStudentRequest

API Response:

StudentResponse

Example

```
public class CreateStudentRequest {  
    private String name;  
    private String email;  
    private String course;  
}
```

Why DTOs?

- Control what clients can send
- Control what clients can receive
- Prevent exposing internal fields
- Support API evolution
- Separate API contract from database structure

Validation

APIs Must Validate Input

Bad request:

```
{ "name": "", "email": "abc", "course": "" }
```

Validation rules

@NotBlank

private String name;

@NotBlank @Email

private String email;

@NotBlank

private String course;

Controller:

@PostMapping("/students")

public Student createStudent(

@Valid @RequestBody CreateStudentRequest request) {

...

}



Never assume that client input is valid.

Error Responses

Don't Return Random Error Messages

Bad:

Something went wrong

Better:

```
{
  "status": 400,
  "message": "Validation failed",
  "timestamp": "2026-08-10T20:00:00",
  "path": "/students"
}
```

For not found:

```
{
  "status": 404,
  "message": "Student with id 101 not found",
  "path": "/students/101"
}
```

📌 Make errors useful to API consumers.

Spring Boot REST Controller

Basic API Controller

```
@RestController
@RequestMapping("/students")
public class StudentController {

    @GetMapping
    public List<Student> getStudents() {
        return studentService.getStudents();
    }

    @GetMapping("/{id}")
    public Student getStudent(@PathVariable Long id) {
        return studentService.getStudent(id);
    }

    @PostMapping
    public Student createStudent(
        @RequestBody Student student) {
        return studentService.createStudent(student);
    }

    @PutMapping("/{id}")
    public Student updateStudent(
        @PathVariable Long id,
        @RequestBody Student student) {
        return studentService.updateStudent(id, student);
    }

    @DeleteMapping("/{id}")
    public void deleteStudent(@PathVariable Long id) {
        studentService.deleteStudent(id);
    }
}
```

- ❑ The controller should handle HTTP concerns. It should not contain all business logic.

Separation of Concerns

Layered Application Design

1

Controller

Handles: HTTP request, Path variables, Query parameters, Request body, HTTP response

2

Service

Handles: Business rules, Calculations, Decisions, Application logic

3

Repository

Handles: Database access, Queries, Persistence

4

Database



Why Not Put Everything in Controller?

Bad Design

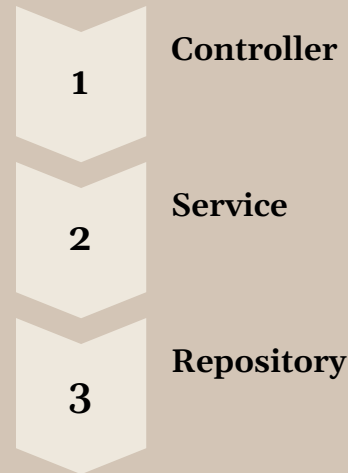
```
@PostMapping("/students")
public Student createStudent(...) {

    // validation
    // business logic
    // database query
    // duplicate checking
    // save
    // response creation
}
```

Problems

- Difficult to test
- Difficult to maintain
- Difficult to reuse
- Controller becomes huge
- Business logic becomes mixed with HTTP logic

Better



Each layer has one major responsibility.

Statelessness

REST APIs Should Be Stateless

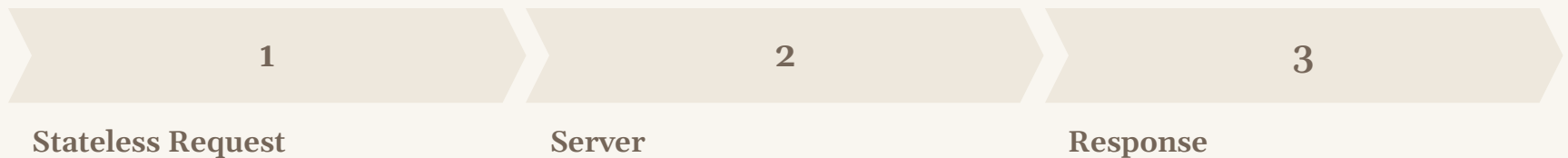
Each request should contain the information required to process it.

Example

Request 1: GET /students/101

Request 2: GET /students/102

The server should not need to remember: "The client previously requested student 101."



Every request can be independently processed.

Why Statelessness Matters

Stateless APIs Are Easier to Scale

Imagine:

Client | Load Balancer |

| | |

Server 1 Server 2 Server 3

Any request can go to any server.

Benefits

- Easier horizontal scaling
- Easier load balancing
- Easier recovery
- Less server-side session dependency
- Better cloud/container compatibility

📌 Stateless does NOT mean: "The application has no state." The database can still store state.

Idempotency

What Does Idempotent Mean?

An operation is idempotent when repeating the same operation produces the same final state.

Example

```
PUT /students/101  
{ "name": "Rahul", "course": "Java" }
```

Send once:

Student 101 → Java

Send again:

Student 101 → Java

Final state remains the same.

HTTP Method Idempotency

Method	Generally Idempotent?
GET	Yes
PUT	Yes
DELETE	Yes
POST	No
PATCH	Depends on implementation

- ❏ "Idempotent" does not mean: The response must always be identical. It means: Repeating the operation should result in the same intended resource state.

POST and Duplicate Requests

Why POST Is Generally Not Idempotent

```
POST /students  
{ "name": "Rahul", "course": "Java" }
```

First request:

Student 101 created

Second request:

Student 102 created

Result

Two resources may be created.

Real-world problem

What if the client doesn't receive the response because of a network failure?

It may retry.

This introduces the need for:

- Idempotency keys
- Duplicate detection
- Safe retry strategies

Consistent API Design

Consistency Is More Important Than Cleverness

Bad

- /students
- /student
- /getStudents
- /studentAPI

Good

- /students
- /students/{id}

Consistent naming

Use one convention across the entire API.

For example:

- /students
- /courses
- /teachers
- /departments

Avoid mixing:

- /students
- /courseList
- /getTeachers
- /DepartmentAPI

Pagination

Never Return Millions of Records

Imagine: 10,00,000 students

Bad: GET /students returning everything.

Problems

Large response

High memory usage

Slow database query

Network overhead

Slow frontend

Poor user experience

Pagination API

Better

```
GET /students?page=0&size=20
```

Meaning:

- page = 0
- size = 20

Return:

```
{
  "content": [
    { "id": 101, "name": "Rahul" }
  ],
  "page": 0,
  "size": 20,
  "totalElements": 1250,
  "totalPages": 63
}
```

📌 Pagination is both an API design concern and a database performance concern.

Filtering

Allow Clients to Request Relevant Data

Instead of:

```
GET /students
```

and filtering everything on the frontend:

Use:

```
GET /students?course=java
```

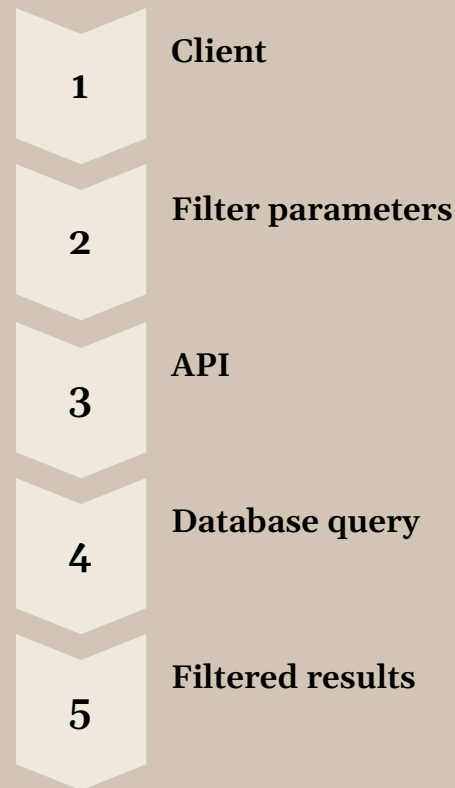
Other examples:

```
GET /students?course=SpringBoot
```

```
GET /students?department=CSE
```

```
GET /students?status=ACTIVE
```

Server-side filtering



Sorting

Let Clients Control Ordering

```
GET /students?sort=name
```

Descending:

```
GET /students?sort=name,desc
```

Multiple sorting criteria:

```
GET /students?sort=course,asc&sort=name,asc
```

📌 The API should define which fields can be sorted. Don't blindly allow arbitrary database fields.

Combining API Features

Realistic Student Search API

```
GET /students  
  ?course=java  
  &status=ACTIVE  
  &sort=name,asc  
  &page=0  
  &size=20
```

This single API supports:

Filtering

Sorting

Pagination

📌 Keep the API predictable. Clients should be able to combine supported query parameters.

API Design Checklist

Before Creating an Endpoint Ask:

1	Resource What resource am I exposing?
2	URI Is the URL resource-oriented?
3	HTTP method Am I using the correct method?
4	Request What data does the client need to send?
5	Response What should the client receive?
6	Status What HTTP status should be returned?
7	Validation What input is invalid?
8	Errors How will errors be represented?
9	Scalability Will this work with large datasets?
10	Consistency Does this follow the same conventions as the rest of the API?

Student Management API

Final API Contract

Operation	Method	Endpoint
Get all students	GET	/students
Get student	GET	/students/{id}
Create student	POST	/students
Replace student	PUT	/students/{id}
Update part of student	PATCH	/students/{id}
Delete student	DELETE	/students/{id}

Query examples

GET /students?page=0&size=20

GET /students?course=java

GET /students?sort=name,asc